Algoritmos y Estructuras de Datos

Trabajo Práctico: LRU Cache

En este trabajo práctico construiremos un TDA implementando la estructura de datos LRU Cache. La “**L**east **R**ecently **U**sed” Cache se basa en el principio de que el dato que fue accedido más recientemente es más probable que vuelva a ser accedido en el corto tiempo. Esto a su vez conlleva a que cuando la memoria (caché) de este TDA se llene (se define un tamaño limitado para ella), se descarte el dato menos usado recientemente (de aquí su nombre), asegurando que los datos más relevantes se encuentren siempre disponibles.

Las primitivas del TDA deben ser:

- crear_lrucache: debe recibir por parámetro una capacidad máxima y una función de comparación de claves, y definir los valores necesarios para inicializar la caché. La función de comparación debe devolver un valor negativo, cero o positivo según la primera clave sea menor, igual o mayor que la segunda.
- agregar_lrucache: agrega un nuevo elemento a la caché. El elemento es agregado al inicio del set de datos, y si, con el agregado de este nuevo elemento, la caché se desborda, debe eliminar el último elemento. Si el dato ya existía (misma clave), es reemplazado y queda al principio del set de datos.
- obtener_lrucache: busca un elemento en la caché por clave. Si lo encuentra, debe moverlo al principio del set de datos y devolver el valor del mismo. Si no lo encuentra devuelve que no existe.
- borrar_lrucache: Elimina un elemento de la caché buscando por clave y lo borra del set de datos.
- vaciar_lrucache: Vacía el set de datos y libera toda la memoria.

Parte 1

Implementar el TDA t_lru_cache utilizando una lista simplemente enlazada.
La implementación de la misma debe ser totalmente genérica, permitiendo que la caché almacene datos de cualquier tipo y tamaño, con claves de cualquier tipo de dato.

Se espera que la organización del proyecto esté bien estructurada con los correspondientes archivos de headers (.h) y de código (.c).

Parte 2

En la implementación de la Parte 1, para obtener, agregar o borrar un elemento es necesario recorrer la lista buscando la clave, por lo que el costo de cada operación crece con la capacidad de la caché.

Implementar una segunda versión del TDA t_lru_cache, con las mismas primitivas, que combine dos estructuras:

- una lista doblemente enlazada, que mantiene los elementos ordenados según su uso (al principio el usado más recientemente y al final el menos usado), y que permite mover un elemento al principio y eliminar el último sin recorrerla;

- un árbol binario de búsqueda ordenado por clave, que funciona como índice: cada nodo del árbol referencia al nodo de la lista que contiene el elemento con esa clave.

De esta manera, la búsqueda por clave se realiza en el árbol y el orden de uso se mantiene en la lista. Ambas estructuras deben mantenerse consistentes en todo momento: cada vez que un elemento se agrega, se reemplaza, se descarta por desborde o se borra, deben actualizarse las dos.

Además de las primitivas del TDA, se debe implementar una primitiva que informe la altura del árbol índice.

La implementación debe seguir siendo totalmente genérica. Se debe prestar especial atención a la liberación de memoria: los datos son referenciados desde ambas estructuras, y ninguno debe quedar sin liberar ni liberarse dos veces.

Parte 3

Una nueva red social llamada “Tweener”, caracterizada por solo admitir posteos de texto de menos de 140 caracteres, está siendo implementada por sus desarrolladores, y quieren validar la eficacia en el uso de una LRU Cache para mostrar el feed (muro) de sus clientes. Para eso deciden hacer una simulación de uso de la aplicación y extraen un lote de pruebas de 50000 posteos que están almacenados en un archivo binario con la estructura `{id: int, texto: char[141]}` donde el id del posteo coincide con el número de registro (la posición relativa del registro en el archivo) lo que facilita las búsquedas directas. Y, además, construyen un archivo binario de no menos de 200 registros que contiene `{usuario: char[15], feed:int[20]}`, donde el feed contiene una lista de los id de posteos que ese usuario ve en su feed. Luego, crean un programa que procesa cada uno de los usuarios, buscando para cada uno todos los posteos de su feed en la base de datos de posteos. Los datos no se muestran por pantalla ni imprimen en ningún archivo ni quedan almacenados en memoria, sino que lo único que se hace es medir los tiempos de proceso del programa para obtener 1 posteo, procesar 1 usuario y el total que demoró para la simulación completa. Por último, se toma ese mismo programa y se le agrega una LRU cache, primero con la implementación de la Parte 1 y luego con la de la Parte 2, para ver en cuánto tiempo mejoró (o no) el procesamiento del programa en cada caso.

Como el tiempo de obtener un único posteo puede ser menor que la resolución del reloj, los tiempos por posteo y por usuario se deben calcular como promedio a partir del tiempo total de la simulación. Además de los tiempos, en cada ejecución con caché se debe informar la cantidad de aciertos (posteos encontrados en la caché), la cantidad de fallos y la cantidad de lecturas realizadas al archivo de posteos.

Se pide construir el programa de simulación mostrando los resultados de manera clara y generar los lotes de prueba de manera que, al ejecutar el programa con distintos lotes y parámetros, se generen distintas situaciones que muestren cómo varía la velocidad de procesamiento con el uso de la caché. El programa que genera los lotes debe recibir una semilla para el generador de números aleatorios, de manera que un mismo lote pueda

volver a generarse. Como mínimo, se deben generar lotes que representen las siguientes situaciones:

- lote concentrado: muchos usuarios ven los mismos posteos;
- lote disperso: los usuarios ven posteos mayormente diferentes;
- lote ordenado: los id de los feeds aparecen en orden creciente. En este caso se debe informar la altura del árbol índice.

Cada lote se debe ejecutar con distintos tamaños de caché.

Junto con el programa se debe entregar un informe breve, de no más de 3 páginas, que presente los resultados obtenidos en tablas o gráficos y responda, al menos, las siguientes preguntas:

- ¿En qué situaciones el uso de la caché mejora el tiempo de procesamiento y en cuáles no?
- ¿A partir de qué capacidad deja de convenir la implementación de la Parte 1? ¿Por qué?
- ¿Qué ocurre con el árbol índice en el lote ordenado y cómo afecta al rendimiento?

Entrega y defensa

Este trabajo práctico se realiza en equipos de hasta 4 personas, sin excepción. Los equipos deben ser informados por la plataforma MIeL en un mensaje en el foro “Trabajo Práctico” con plazo máximo hasta el día **20/10/2026 a las 23:59.**

La entrega del trabajo práctico se realiza de manera individual. Cada integrante del grupo debe enviar desde la sección prácticas de MIeL en la práctica “Trabajo Práctico” a todos los docentes, una copia del programa y del informe, informando nuevamente en el mensaje los integrantes de su grupo.

El plazo **máximo** de entrega del trabajo práctico será el día **02/11/2026 a las 19:00** a través de la plataforma MIeL y habrá una defensa del mismo en la clase del día **02/11/2026**, contando con una evaluación grupal e individual. Durante la defensa, cada equipo deberá implementar en el momento una funcionalidad adicional sobre su propio trabajo.

Los trabajos entregados después de dicha fecha, no serán tenidos en cuenta, por lo que dicho equipo tendrá un ausente en la entrega y en la defensa, teniendo que utilizar la fecha de recuperatorio para regularizar su situación.